BusinessIntelligence.AI---BID

BID - Business Investigation Department

A decision-intelligence platform that turns fragmented business data into explainable KPI investigations, evidence-backed root causes, persona-specific business narratives, ranked actions, feedback signals, and auditable reports.

📋 Table of Contents

🎯 Overview

BID is an intelligent business analytics platform that:

- Ingests raw business data (orders, marketing metrics, KPIs)
- Detects anomalies in key performance indicators using statistical methods
- Performs root cause analysis on identified anomalies
- Generates AI-driven business narratives (persona-based insights)

- Retrieves supporting evidence from customer reviews/tickets
- Creates professional PDF reports with actionable recommendations
- Learns from analyst feedback to improve recommendations over time when sufficient historical feedback is available

---

🧭 BusinessIntelligence.ai Round 2 — BID Business & Solution Narrative

1. Executive Summary

BID (Business Investigation Department) is designed around a simple business problem: organizations can see that a KPI has moved, but the difficult and time-consuming work begins after the dashboard turns red.

A revenue decline, conversion drop, rising customer-acquisition cost, or order-volume change rarely has one isolated cause. The explanation may be distributed across marketing activity, customer behaviour, product mix, operational performance, and supporting qualitative feedback. Analysts therefore spend significant time joining sources, checking definitions, comparing historical baselines, validating possible drivers, searching customer evidence, and translating the findings into actions for different business stakeholders.

BID converts that investigation into a structured intelligence-to-action workflow:

```
Fragmented Business Sources
        ↓
Data Reconciliation
        ↓
KPI / Semantic Context
        ↓
Material Movement Detection
        ↓
Driver & Root-Cause Analysis
        ↓
Confidence / Uncertainty
        ↓
Evidence Retrieval
        ↓
Persona-Specific Interpretation
        ↓
Ranked Business Actions
        ↓
Analyst Feedback
        ↓
Improved Decision Support
```

The central design principle is that **quantitative business truth is established before an LLM is asked to communicate it**. Statistical detection, forecasting, KPI calculations, contribution analysis, confidence scoring, database queries, access decisions, and recommendation scoring are handled by deterministic or ML components. LLMs are used where language and contextual synthesis add value: explaining structured findings, adapting the narrative to a persona, generating contextual action wording, and supporting conversational interaction.

This separation makes BID more explainable, reproducible, auditable, and suitable for governed business decision support than an architecture that asks a general-purpose LLM to infer business numbers directly.

2. The Business Problem

Traditional BI systems are excellent at answering:

> **What happened?**

They are much less effective at answering the complete decision question:

> **What happened, why did it happen, how confident are we, what evidence supports the explanation, who should act, and what should they do next?**

Consider a situation where revenue falls by 12%.

A conventional dashboard may show the revenue trend and the percentage decline. An analyst still needs to investigate:

- Did visitor volume change?
- Did conversion rate change?
- Did order volume change?
- Did AOV change?
- Did acquisition cost or advertising behaviour change?
- Was the movement concentrated in particular products?
- Do customer reviews or support interactions indicate a related issue?
- Is the evidence strong enough to call one driver the primary cause?
- Should a Marketing Manager and a Sales Operations Manager receive the same explanation?
- Which business action is most appropriate?
- Can the organization learn from the eventual outcome?

BID is designed to answer these questions in one decision workspace.

Why the problem is difficult

Real organizations commonly face:

- fragmented source systems
- different data grains
- different refresh cadences
- inconsistent KPI definitions
- interacting business drivers
- sparse history for new products or KPIs

- contradictory qualitative evidence
- role-specific decision rights
- large numbers of possible actions
- pressure to explain AI-generated decisions
- cost and latency constraints for LLM usage.

BID addresses these as a connected investigation rather than treating each issue as an isolated analytics feature.

3. From Dashboard BI to Decision Intelligence

Traditional BI	BID Decision Intelligence
Shows KPI trends	Detects material KPI movements
Shows historical values	Compares movement against relevant baselines
Requires manual investigation	Structures driver investigation
Treats KPIs independently	Connects related KPIs and business drivers
Limited qualitative context	Retrieves supporting customer evidence
One generic explanation	Persona-specific business interpretation
Analyst chooses actions manually	Ranks practical business actions
Limited uncertainty communication	Provides confidence and abstention signals
Static reporting	Interactive investigation + PDF reporting
Little learning from outcomes	Captures analyst/action/outcome feedback

The product therefore sits between analytics, business investigation, and decision support.

4. End-to-End BID Architecture

Data Layer

BID can work with structured business datasets representing:

- orders and transactions
- marketing activity

- KPI time series
- customer reviews and support evidence
- PostgreSQL business data
- uploaded KPI datasets.

The prototype uses portable file-based ingestion where appropriate, while the architecture keeps the analytical layer independent from a single ingestion mechanism.

Intelligence Layer

The intelligence pipeline performs:

1. data validation and reconciliation
2. KPI calculation and semantic interpretation
3. anomaly detection
4. forecasting-based validation
5. driver comparison
6. contribution analysis
7. confidence assessment
8. evidence retrieval
9. recommendation generation and ranking.

Language Layer

The LLM sits after structured analysis.

It receives the computed business context and converts it into human-readable output for the relevant persona.

This means:

```
Raw Data
   ↓
Deterministic Analytics
   ↓
Structured Business Findings
   ↓
LLM Contextualisation
   ↓
Human-Readable Decision Support
```

rather than:

```
Raw Data
   ↓
LLM Guess
   ↓
Business Decision
```

5. KPI Intelligence and Semantic Contracts

A KPI is more than a column of numbers. BID treats KPI meaning as business metadata.

A KPI can be associated with:

Semantic Attribute	Purpose
Name	Identifies the business metric
Definition	Establishes the business meaning
Formula	Defines calculation logic
Unit	Defines interpretation of values
Drivers	Identifies upstream factors
Drives	Identifies downstream business outcomes
Threshold	Defines material movement

Direction of good	Indicates whether higher/lower is desirable
Source	Establishes data provenance
Lineage	Explains how the KPI was produced
Access policy	Supports role-aware governance

This prevents the same KPI from being interpreted differently across teams and creates a foundation for reusable KPI intelligence across industries.

6. Connected KPI Intelligence

BID is designed to investigate KPIs as a connected business system.

A representative KPI chain is:

```
Visitors
    ↓
Conversion Rate
    ↓
Orders
    ↓
Revenue
    ↓
AOV / Revenue Efficiency
```

Marketing efficiency can provide another relationship:

```
Ad Spend
    ↓
CAC / Acquisition Efficiency
    ↓
Traffic & Conversion
    ↓
Orders
    ↓
Revenue
```

This allows the system to move beyond:

> "Revenue decreased."

towards a structured investigation such as:

> "Revenue decreased alongside a decline in order volume and conversion, while changes in acquisition efficiency and product contribution provide additional explanatory context."

The exact explanation is determined from the available data rather than being invented by the language model.

7. Material KPI Movement Detection

BID uses statistical methods to identify unusual KPI behaviour.

The current implementation includes:

- rolling Z-score analysis
- configurable thresholds
- historical baselines
- Prophet-based time-series forecasting
- combined anomaly interpretation.

The purpose is not simply to flag every fluctuation.

The goal is to identify movements worth investigating and then connect those movements to explanatory business factors.

8. Root Cause and Contribution Analysis

BID separates anomaly detection from root-cause investigation.

Detection

> Something unusual happened.

Investigation

> Which available factors changed materially around the same time?

Contribution

> Which factors provide the strongest quantitative explanation?

The root-cause engine can use:

- baseline comparison
- percentage change
- driver relationships
- factor correlation
- multi-KPI overlap
- product-level contribution where available
- confidence scoring.

For revenue investigations, product-level analysis can compare product performance against a trailing baseline and expose contribution, volume, price and related effects.

This creates a traceable chain from KPI movement to candidate drivers.

9. Multi-Factor Business Reasoning

Business outcomes rarely depend on one variable.

For example:

```
Revenue ↓
   ├── Orders ↓
   ├── Conversion Rate ↓
   ├── Visitor Mix changed
   ├── CAC changed
   └── Product contribution shifted
```

BID can examine these signals together.

The system therefore supports a multi-factor explanation rather than forcing a single-cause story.

This is especially valuable for executives and operators because the most useful explanation is often:

> "Several factors contributed, with one dominant driver and supporting secondary drivers."

rather than:

> "Factor X caused everything."

10. Confidence, Uncertainty and Abstention

BID treats uncertainty as a first-class part of the decision.

Confidence can consider factors such as:

- available historical depth
- strength and number of drivers
- consistency between signals
- supporting evidence
- competing explanations
- data availability.

The output can therefore distinguish between:

High-confidence investigation

Strong quantitative and contextual support exists.

Lower-confidence investigation

Evidence is weaker, history is limited, or multiple explanations compete.

Abstention

The system does not force a definitive root cause when the available evidence is insufficient.

This is important because a responsible business intelligence system should know when the evidence does not justify certainty.

11. Sparse-History Intelligence

New KPIs, new products, and new business processes may not have enough historical observations for reliable statistical inference.

BID's confidence architecture supports this situation by reducing certainty when history is insufficient and making the limitation visible to the decision-maker.

This prevents the system from presenting a statistically weak conclusion with the same confidence as a well-supported historical pattern.

The same architecture can later be extended with richer cold-start strategies as more business history becomes available.

12. Evidence Grounding and RAG

Quantitative analysis tells BID and which factors are associated with the movement.

Customer evidence provides another layer:

> **What is happening in the business environment around that movement?**

BID can retrieve supporting customer reviews or feedback using relevance and temporal context.

The evidence pipeline considers:

- root-cause context
- relevant factors
- semantic similarity
- anomaly timing
- evidence date/source.

Temporal relevance is important because a review from months ago may be semantically similar but unrelated to a current event.

The result is a grounded investigation:

```
Quantitative Signal
      +
Customer / Business Evidence
      ↓
Contextual Business Explanation
```

13. Persona-Aware Decision Support

Different decision-makers control different levers.

Marketing Manager

The Marketing Manager is primarily interested in:

- traffic
- conversion
- CAC
- acquisition efficiency
- campaign effectiveness
- marketing spend
- channel behaviour.

The narrative therefore prioritises marketing-controlled drivers and actions.

Sales Operations Manager

The Sales Operations Manager is more concerned with:

- order volume
- AOV
- product mix
- operational efficiency
- fulfilment-adjacent factors
- conversion and pipeline effects.

The same KPI movement can therefore produce different decision support without changing the underlying quantitative truth.

14. Recommendation and Action Intelligence

BID is designed to move from explanation to action.

The recommendation pipeline can be represented as:

```
Observed Driver
      ↓
Controllable Business Lever
      ↓
Candidate Action
      ↓
Effectiveness / Ranking Signal
      ↓
Persona-Relevant Recommendation
      ↓
Outcome Feedback
```

The system combines business action definitions, deterministic context, historical feedback signals, and ML-based ranking where available.

LLM-generated actions can add contextual wording, but the underlying quantitative context comes from the analytical pipeline.

A strong recommendation should answer:

- What should be changed?
- Why this lever?
- Who can act?
- What business outcome is expected?
- What should be monitored afterwards?

15. Feedback Learning

BID includes a feedback loop around business decisions.

Feedback can capture:

- analyst rating
- whether an action was taken
- outcome
- outcome value

- root-cause context
- KPI movement context.

The recommendation layer can use historical feedback to improve action scoring.

The current prototype demonstrates feedback-aware recommendation ranking on the available decision variables. The architecture is intentionally structured so that additional feedback dimensions, evaluation metrics, outcome learning, and broader retraining can be incorporated as the decision history grows.

16. Custom KPI Extensibility

BID supports user-defined KPIs through metadata such as:

- KPI name
- business definition
- formula
- unit
- drivers
- downstream relationships
- direction of good
- threshold.

A derived KPI can use variables originating from different daily datasets.

Conceptually:

```
Source Dataset A ─┐
                  ├── Calculation Context → Derived KPI
Source Dataset B ─┘
```

For example:

```
KPI dataset:
date, revenue, aov

Additional daily dataset:
date, visitors, orders, cac

Derived KPI:
Revenue Per Visitor = revenue / visitors
```

The important semantic distinction is that source variables are calculation inputs; the resulting derived metric is the business KPI presented for analysis.

This creates a foundation for a governed KPI builder that can later support more enterprise data sources and richer semantic relationships.

17. Evidence, Lineage and Auditability

BID is designed so that an important conclusion can be traced through:

```
Source
  ↓
Data Transformation
  ↓
KPI
  ↓
Anomaly Method
  ↓
Driver Analysis
  ↓
Confidence
  ↓
Evidence
  ↓
Narrative
  ↓
Recommendation
```

This allows a business user or evaluator to distinguish:

- source data
- computed metrics
- analytical methods
- retrieved evidence
- LLM-generated language.

That separation is central to trustworthy AI-assisted BI.

18. Deterministic Analytics vs LLM

Processing Layer	Primary Mechanism	Role
Data loading	Python / Pandas / SQL	Structured data preparation
KPI calculation	Deterministic calculations	Quantitative truth
Anomaly detection	Z-score	Statistical movement detection
Forecasting	Prophet	Time-series expectation
Root cause	Deterministic driver analysis	Quantitative explanation
Product contribution	Database + analytics	Product-level explanation
Confidence	Rules/statistical signals	Uncertainty assessment
Evidence retrieval	Retrieval / similarity	Supporting context
Recommendation scoring	Rules + ML + feedback	Action ranking
Narrative	LLM	Human-readable synthesis
Persona adaptation	LLM + configuration	Role-specific communication
Conversation	LLM + deterministic tools	Natural-language interaction

This hybrid design deliberately assigns each task to the technology best suited to it.

19. Security and Governance

BID includes a persona-aware configuration layer that can express access around:

- KPIs

- sources
- features
- business personas.

This creates a foundation for enterprise governance where different decision-makers receive the information required for their role.

The same architecture can be expanded toward:

- row-level security
- column-level security
- domain-level restrictions
- tenant isolation
- audit logging
- centralized identity providers.

The backend remains the appropriate enforcement boundary while the frontend can provide a role-aware user experience.

20. Runtime Telemetry and AI Economics

AI systems must be evaluated not only on answer quality but also on operational efficiency.

BID exposes runtime telemetry including:

- total analysis latency
- LLM call count
- models used
- token usage
- per-request latency
- estimated LLM cost.

This makes it possible to evaluate:

```
Insight Quality
+
Latency
+
Token Efficiency
+
Estimated Cost
```

The estimated cost is an engineering estimate based on model usage; actual provider billing remains the authoritative source for production accounting.

Future optimization can include:

- caching
- prompt compression
- model selection
- asynchronous execution
- provider substitution
- request batching.

21. Decision Workspace

The React workspace is designed as a business investigation environment rather than a collection of disconnected charts.

Users can move between:

- KPI selection
- anomaly analysis
- visualisation
- root cause
- recommendations
- custom KPI creation
- conversational assistance
- PDF reporting.

This supports an investigation flow in which the user can move from observation to explanation to action without leaving the decision context.

22. End-to-End Business Investigation Example

Situation

Revenue shows a material decline.

Investigation

1. BID detects the unusual movement using statistical analysis.
2. Related KPIs are inspected.
3. Orders, conversion and other available drivers are compared against historical baselines.
4. Driver contribution ranks the strongest explanatory factors.
5. Product-level analysis provides additional context where available.
6. Relevant customer evidence is retrieved.
7. Confidence is calculated from the available signals.
8. The Marketing Manager receives an acquisition-focused narrative.
9. The Sales Operations Manager receives an operational/product-focused narrative.
10. Candidate actions are generated and ranked.
11. The user can record feedback about the recommendation and eventual outcome.
12. Runtime telemetry records the computational and LLM footprint.
13. The same structured analysis can be rendered into a PDF report.

Result

Instead of receiving a dashboard alert, the business receives a structured investigation:

```
WHAT CHANGED?
Revenue moved materially.

WHY?
Several related drivers changed, with ranked quantitative contribution.

HOW CONFIDENT?
Confidence is explicitly communicated.

WHAT EVIDENCE?
Supporting business/customer evidence is surfaced.

WHO SHOULD ACT?
The explanation is adapted to the decision-maker.

WHAT NEXT?
Ranked, driver-grounded actions are provided.
```

23. Business Value

BID creates value across the complete investigation lifecycle.

Faster time-to-insight

Automating repetitive investigation steps can reduce the time analysts spend moving between dashboards, databases and evidence sources.

Better analyst productivity

Analysts can focus on validating decisions and business context instead of repeatedly assembling the same investigation manually.

More consistent decision-making

Semantic KPI definitions, structured root-cause logic and recommendation scoring create a repeatable investigation framework.

Better explainability

Every major conclusion can be separated into quantitative findings, evidence and generated language.

Role-aware decisions

Different business owners can receive explanations aligned with their decision rights and controllable levers.

Organizational learning

Feedback creates a mechanism for capturing which recommendations were useful and what happened after actions were taken.

24. Target Users

Primary users

- Business analysts
- Marketing managers
- Sales operations managers
- Revenue managers
- Operations leaders
- BI teams.

Secondary users

- Product managers
- Finance teams
- Customer experience teams
- Executives requiring concise KPI investigations.

25. Industry Applicability

The prototype is demonstrated using an e-commerce-style KPI ecosystem because it provides a clear connected KPI chain.

The same architecture can support:

- retail
- SaaS
- subscription businesses

- marketplaces
- digital marketing
- consumer products
- operations
- revenue management.

The analytical core is designed to remain reusable while KPI semantics, business drivers, personas and action catalogs can be configured for the specific industry.

26. Prototype Scope and Practical Limitations

The current prototype deliberately uses a controlled and reproducible environment so that the complete intelligence-to-action workflow can be demonstrated clearly.

Current operating assumptions include:

- representative structured business datasets
- controlled CSV ingestion for portable demonstrations
- bounded historical data
- a finite evidence corpus
- hosted LLM services
- feedback learning based on currently available decision variables.

These choices make the prototype easier to reproduce and evaluate.

The architecture is designed for expansion toward:

- governed warehouse connectors
- enterprise APIs
- larger evidence stores
- broader KPI catalogs
- richer feedback dimensions
- enterprise identity and access management
- continuous evaluation
- larger-scale deployment.

27. Scalability and Expansion

BID is intentionally modular so that the prototype can evolve without replacing the core intelligence pipeline.

Stage 1 — Decision Intelligence

- KPI anomaly detection
- root cause analysis
- evidence grounding
- persona narratives
- recommendations
- feedback
- reporting.

Stage 2 — Enterprise Hardening

- broader connectors
- stronger entitlement policies
- richer observability
- caching
- performance optimisation
- governed data access.

Stage 3 — Enterprise Scale

- multi-tenant deployment
- warehouse-native processing
- enterprise connectors
- alerting integrations
- larger evidence repositories
- cross-business KPI catalogs.

Stage 4 — Advanced Intelligence

- causal inference
- scenario simulation
- proactive anomaly alerts
- action outcome prediction
- continuous model evaluation

- data/model drift monitoring.

The important architectural principle is that these are extensions around the same decision-intelligence core rather than a replacement of the product.

28. Risks and Mitigations

Risk	Business Impact	BID Mitigation	Expansion Path
Data quality	Incorrect analysis	Validation and deterministic processing	Data-quality monitoring
Sparse history	Weak statistical confidence	Confidence degradation and cautious interpretation	Cold-start strategies
False positives	Analyst fatigue	Thresholds + multiple detection methods	Adaptive thresholds
Contradictory evidence	Misleading narrative	Confidence/evidence separation	Evidence conflict resolution
LLM dependency	Availability/cost risk	Deterministic core + controlled LLM use	Provider abstraction/fallbacks
Token growth	Higher operating cost	Runtime telemetry	Caching and prompt optimisation
Model drift	Recommendation degradation	Feedback/evaluation architecture	Continuous evaluation
Data drift	Changing KPI behaviour	Baseline/forecast monitoring	Automated drift detection
Schema variation	Integration effort	Validation and configurable inputs	Connector framework
Security	Unauthorized information access	Persona-aware governance architecture	Enterprise IAM/RLS/CLS

29. BusinessIntelligence.ai Round 2 Requirement Alignment

Round 2 Requirement	BID Capability	Technical Mechanism	Business Value
Detect material KPI movements	Statistical anomaly detection	Rolling Z-score + Prophet	Finds business events worth investigating
Reconcile heterogeneous sources	Multi-source data layer	Pandas + PostgreSQL + structured ingestion	Creates a consistent analytical context
Identify explanatory drivers	Root cause engine	Baselines, changes, correlations, contribution analysis	Converts alerts into explanations
Persona-specific narratives	Role-aware narratives	Persona configuration + LLM synthesis	Makes insights actionable for each decision-maker
Traceable evidence	Evidence/RAG layer	Relevance + temporal retrieval	Grounds explanations in supporting context
Communicate uncertainty	Confidence/abstention	Deterministic confidence signals	Reduces overconfident decisions
Recommend practical actions	Recommendation engine	Rules + ML ranking + feedback + contextual actions	Moves from insight to execution
Learn from feedback	Decision feedback loop	Ratings, actions and outcomes	Creates organizational learning
Security	Persona-aware entitlement architecture	KPI/source/feature policies	Supports governed enterprise deployment

Cost and latency	Runtime telemetry	Latency, calls, tokens, estimated cost	Makes AI economics observable
Scalability	Modular architecture	Configurable services and semantic layer	Supports future enterprise expansion

Minimum Prototype Coverage

Expected Prototype Element	BID Demonstration
3–5 connected KPIs	Revenue, conversion, AOV, visitors, orders/CAC relationships
2–3 data sources	Structured KPI/business data, PostgreSQL, customer evidence
KPI semantic contract	Definitions, formulas, drivers, thresholds and direction-of-good
At least two personas	Marketing Manager and Sales Operations Manager
Multi-factor movement	Connected KPI + driver contribution analysis
Low-confidence scenario	Confidence and abstention framework
Sparse-history scenario	History-aware confidence handling
Role-aware security	Persona-aware access architecture
Evidence and lineage	Source, method, contribution, confidence and lineage context
LLM vs non-LLM	Explicit deterministic/LLM processing separation
Runtime telemetry	Latency, model calls, tokens and estimated cost

30. Why BID Is More Than an AI Dashboard

BID's value is not the presence of an LLM.

The core innovation is the orchestration of multiple intelligence layers into a single business investigation:

```
Statistics
    +
Forecasting
    +
Business Semantics
    +
Driver Contribution
    +
Customer Evidence
    +
Persona Context
    +
Recommendation Ranking
    +
Feedback
    +
LLM Communication
```

The result is a system that attempts to close the gap between:

"The metric changed."

and:

"Here is the evidence-backed explanation, here is how confident we are, here is who should care, and here is what they can do next."

✨ Features

- **Data Management**: PostgreSQL database for storing raw data, marketing metrics, and KPIs
- **Multi-Method Anomaly Detection**: Z-score analysis + Prophet forecasting for stronger detection
- **Root Cause Analysis**: Driver-based analysis to identify factors contributing to anomalies
- **AI-Powered Narratives**: Persona-based business insights (Marketing Manager, Sales Ops perspectives)

- **Evidence Retrieval (RAG)**: Supporting evidence from customer reviews and feedback
- **PDF Report Generation**: Professional reports with graphs, tables, and recommendations
- **Confidence Scoring**: Reliability metrics for analysis results
- **Feedback Loop**: Learn from analyst decisions to improve recommendations over time
- **Interactive API Docs**: Auto-generated Swagger UI for testing endpoints
- **Chat Interface**: Conversational AI for Q&A about anomalies
- **ML-Based Action Ranking (v5)**: scikit-learn recommendation ranking with a safe fallback when feedback history is insufficient
- **Dynamic KPI Selection**: Analysis windows use the KPI fields actually present in the uploaded dataset
- **Product-Level Revenue Drivers**: PostgreSQL-backed product contribution analysis for revenue investigations
- **PDF Chat**: Attach a PDF directly in the chat drawer and ask questions about the document with Gemini

---

🛠️ Tech Stack

- **Backend**: Python 3.8+ with FastAPI
- **Database**: PostgreSQL 12+
- **Frontend**: React 19+ with Vite (ES6+)
- **Report Generation**: ReportLab, Matplotlib, Pandas
- **Anomaly Detection**: SciPy (Z-score), Prophet (time-series forecasting)
- **ML Training**: scikit-learn, joblib (for v5 model)
- **LLM Integration**: Groq for conversational/tool-calling flows and Google Gemini for PDF document Q&A
- **Environment Management**: python-dotenv
- **API Server**: Uvicorn (ASGI)
- **Data Processing**: Pandas, NumPy

Language Composition:

- Python: 69.2%
- JavaScript: 23.4%
- CSS: 7.3%

- HTML: 0.1%

📁 Project Structure

```
BusinessIntelligence.AI---BID/
├── main.py                          # FastAPI application (entry point)
├── Create_inputDataBase.py          # Database initialization (core KPI
tables)
├── database.py                      # Database utilities (feedback
table for v5 model)
├── report_pdf.py                    # PDF report generation module
├── anomaly_detector.py              # Anomaly detection (Z-score)
├── prophet_detector.py              # Anomaly detection (Prophet time-
series)
├── root_cause.py                    # Root cause analysis engine
├── product_drivers.py               # Product-level revenue
contribution and profit snapshot
├── action_engine.py                 # Business action recommendations
├── recommendation_engine_v5.py      # ML-based action ranking (requires
3000+ feedbacks)
├── llm_narrative.py                 # AI narrative generation
├── text_retrieval.py                # RAG / evidence retrieval
├── chatbot.py                       # Conversational AI
├── business_config.py               # Configuration for KPIs
├── seedfeedback.py                  # Seed sample feedback data for v5
testing
├── samplefrontend/                  # React + Vite frontend
│   ├── src/
│   ├── package.json
│   └── vite.config.js
├── .env                             # Environment variables (NOT in
version control)
└── requirements.txt                 # Python dependencies
```

🚀 Installation

Prerequisites

- **Python** 3.8 or higher
- **PostgreSQL** 12 or higher (running locally or remotely)
- **Node.js** 16+ (for frontend)
- **pip** or conda (Python package manager)
- **Git**

Backend Setup

1. **Clone the repository**

```
git clone https://github.com/Giridhar692005/BusinessIntelligence.AI---BID.git
cd BusinessIntelligence.AI---BID
```

2. **Create a virtual environment**

```
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

3. **Install Python dependencies**

```
pip install -r requirements.txt
```

Core Python Libraries:

- `fastapi==0.141.1` - Web framework
- `uvicorn==0.52.4` - ASGI server
- `pandas==2.2.3` - Data processing
- `numpy==2.1.2` - Numerical computing
- `scikit-learn==1.6.0` - Machine learning
- `scipy==1.14.1` - Statistical analysis
- `prophet==1.4.0` - Time-series forecasting

- `psycopg2-binary==2.9.12` - PostgreSQL adapter
- `python-dotenv==1.0.1` - Environment variables
- `python-multipart==0.0.32` - File upload handling
- `pydantic==2.13.4` - Data validation
- `matplotlib==3.9.3` - Visualization
- `pillow==11.0.0` - Image processing
- `reportlab==5.0.1` - PDF generation
- `google-genai==2.20.0` - Google Gemini API
- `requests==2.32.3` - HTTP client
- `SQLAlchemy==2.0.52` - Database ORM
- `joblib==1.4.2` - Model serialization (for v5)
- `tenacity==9.1.4` - Retry handling

4. **Configure environment variables**

```
# Create .env file in root directory
# Edit with your configuration (see Configuration section below)
```

Frontend Setup (Optional - for full UI)

1. **Navigate to frontend directory**

```
cd samplefrontend
```

2. **Install Node dependencies**

```
npm install
```

Core JavaScript/React Libraries:

- `react@^19.0.0` - UI framework
- `react-dom@^19.0.0` - React DOM bindings
- `vite@^7.0.0` - Build tool
- `@vitejs/plugin-react@^5.0.0` - Vite React plugin
- `recharts@^3.10.1` - Charting library

- `react-mosaic-component@^7.0.0` - Mosaic layout component

3. **Start development server**

```
npm run dev
```

Frontend will be available at `http://127.0.0.1:5173`

🗄️ Database Setup

Prerequisites

- PostgreSQL running locally or remotely
- Database credentials ready

Configuration

1. **Create `.env` file** in root directory:

```
# Create a .env file with your configuration
```

2. **Configure database and API parameters** in `.env`:

```
# ============ DATABASE CONFIGURATION ============
DB_HOST=localhost
DB_NAME=business_Ai
DB_USER=postgres
DB_PASSWORD=your_secure_password
DB_PORT=5432

# ============ LLM CONFIGURATION ============
# Google Gemini API for AI narratives
GEMINI_API_KEY=your_gemini_api_key_here

# ============ API CONFIGURATION ============
# IMPORTANT: Use GROQ_API_KEY (not API_KEY)
GROQ_API_KEY=your_groq_api_key_for_external_callers

DEBUG=False
LOG_LEVEL=INFO
```

3. **Initialize core database tables** (for basic KPI analysis):

   ```
   python Create_inputDataBase.py --init
   ```

   Creates tables:

   - `RawData`: Order and transaction data
   - `MarketingData`: Ad spend and website visit metrics
   - `Kpis`: Key performance indicators (conversion rate, revenue, AOV)

4. **Initialize feedback database** (for recommendation_engine_v5.py ML training):

   ```
   python database.py --init
   ```

   Creates table:

   - `business_decisions`: Analyst feedback for ML model training (required for v5)

5. **Verify database connection** (optional):

```
python Create_inputDataBase.py
python database.py
```

Database Schema

RawData Table

```
CREATE TABLE RawData (
  order_id         VARCHAR(20) PRIMARY KEY,
  customer_id      VARCHAR(20) NOT NULL,
  product_id       VARCHAR(30) NOT NULL,
  unit_price       NUMERIC(10, 2) NOT NULL,
  quantity         INTEGER NOT NULL,
  order_date       DATE NOT NULL,
  production_cost  NUMERIC(10, 2)
);
```

MarketingData Table

```
CREATE TABLE MarketingData (
  date             DATE PRIMARY KEY,
  ad_spend         NUMERIC(10, 2),
  website_visits   INTEGER
);
```

Kpis Table

```
CREATE TABLE Kpis (
  date             DATE PRIMARY KEY,
  conversion_rate  NUMERIC(6, 4),
  revenue          NUMERIC(12, 2),
  aov              NUMERIC(10, 2)
);
```

Business Decisions Table (for v5 ML model training)

```
CREATE TABLE business_decisions (
  id                          SERIAL PRIMARY KEY,
  kpi                         VARCHAR(50),
  anomaly_date                DATE,
  root_cause                  VARCHAR(200),
  action_id                   VARCHAR(50),
  recommended_action          TEXT,
  analyst_rating              INTEGER,
  action_taken                BOOLEAN,
  outcome                     VARCHAR(100),
  outcome_value               NUMERIC,
  primary_driver_pct_change   NUMERIC,
  confidence_score            NUMERIC,
  visitors_change             NUMERIC,
  orders_change               NUMERIC,
  revenue_change              NUMERIC,
  aov_change                  NUMERIC,
  cac_change                  NUMERIC,
  ad_spend_change             NUMERIC,
  created_at                  TIMESTAMP DEFAULT NOW()
);
```

🚀 Running the Application

Start the FastAPI Backend Server

```
uvicorn main:app --reload --port 8000
```

Parameters:

- `main:app` — loads the FastAPI app from `main.py`
- `--reload` — auto-restarts server on code changes (development only)
- `--port 8000` — runs on `http://127.0.0.1:8000`

Access the API Documentation

Once running, open your browser and navigate to:

```
http://127.0.0.1:8000/docs
```

This displays the **Interactive Swagger UI** where you can:

- View all available endpoints
- Test endpoints with sample data
- See request/response schemas
- Download API specifications

Alternative API docs (ReDoc format):

```
http://127.0.0.1:8000/redoc
```

Health Check

```
curl http://127.0.0.1:8000/
# Response: {"status": "ok", "message": "KPI Anomaly Detection API is
running"}
```

📡 API Endpoints

1. Data Upload Endpoints

Upload Order Data

```
POST /upload-orders
Content-Type: multipart/form-data

Parameters:
  file: CSV file with columns [order_id, customer_id, product_id,
unit_price, quantity, order_date, production_cost]

Response:
{
  "status": "ok",
  "rows_upserted": 150
}
```

Upload Marketing Data

```
POST /upload-marketing
Content-Type: multipart/form-data

Parameters:
  file: CSV file with columns [date, ad_spend, website_visits]

Response:
{
  "status": "ok",
  "rows_upserted": 90
}
```

Upload Customer Reviews

```
POST /upload-reviews
Content-Type: multipart/form-data

Parameters:
  file: CSV file with columns [date, text]

Response:
{
  "status": "ok",
  "rows_loaded": 200
}
```

---

2. KPI Management Endpoints

Calculate KPIs

```
POST /calculate-kpis

Response:
{
  "status": "ok",
  "days_calculated": 90
}
```

Computes conversion_rate, revenue, and AOV from RawData + MarketingData.

Get Calculated KPIs

```
GET /kpis

Response:
[
  {
    "date": "2024-01-15",
    "revenue": 15000.50,
    "conversion_rate": 0.0250,
    "aov": 125.75
  },
  ...
]
```

---

3. Anomaly Detection Endpoints

Basic Anomaly Detection (Z-Score)

```
POST /detect?kpi=revenue&window=14&threshold=2.5
Content-Type: multipart/form-data

Parameters:
  file: CSV with KPI time series
  kpi: Column name to analyze (e.g., "revenue")
  window: Rolling window size in days (default: 14)
  threshold: Z-score cutoff (default: 2.5)

Response:
{
  "kpi": "revenue",
  "window": 14,
  "threshold": 2.5,
  "total_days": 90,
  "anomaly_count": 3,
  "data": [
    {
      "date": "2024-01-15",
      "value": 25000.00,
      "is_anomaly": true,
      "zscore": 3.2
    },
    ...
  ]
}
```

Strong Anomaly Detection (Z-Score + Prophet)

```
POST /detect-strong?
kpi=revenue&window=14&threshold=2.5&interval_width=0.90
Content-Type: multipart/form-data

Parameters:
  file: CSV with KPI time series
  kpi: Column name to analyze
  window: Rolling window size (default: 14)
  threshold: Z-score cutoff (default: 2.5)
  interval_width: Prophet confidence interval width (default: 0.90)

Response:
{
  "kpi": "revenue",
  "prophet_available": true,
  "window": 14,
  "threshold": 2.5,
  "interval_width": 0.90,
  "total_days": 90,
  "anomaly_count": 4,
  "zscore_only_count": 2,
  "prophet_only_count": 1,
  "both_count": 1,
  "data": [
    {
      "date": "2024-01-15",
      "value": 25000.00,
      "is_anomaly": true,
      "detected_by": ["zscore", "prophet"]
    },
    ...
  ]
}
```

Detect All KPIs (Z - Score)

```
POST /detect-all?window=14&threshold=2.5
Content-Type: multipart/form-data

Parameters:
  file: CSV with all KPI columns
  window: Rolling window size (default: 14)
  threshold: Z-score cutoff (default: 2.5)

Response:
{
  "revenue": {
    "anomaly_count": 3,
    "data": [...]
  },
  "conversion_rate": {
    "anomaly_count": 2,
    "data": [...]
  },
  "aov": {
    "anomaly_count": 1,
    "data": [...]
  }
}
```

---

4. Visualization Endpoints

Plot KPI with Anomalies (PNG Image)

```
POST /plot?kpi=revenue&window=14&threshold=2.5
Content-Type: multipart/form-data

Response: PNG image (direct binary)
```

Returns an image showing:

- KPI trend line

- Anomalies marked as black X's
- Historical baseline

Plot KPI as Base64 JSON

```
POST /plot-base64?kpi=revenue&window=14&threshold=2.5
Content-Type: multipart/form-data

Response:
{
  "kpi": "revenue",
  "anomaly_count": 3,
  "image_base64": "..."
}
```

Easier for React frontends to display.

---

5. Root Cause Analysis Endpoints

Analyze Root Causes

```
POST /root-cause?date=2024-01-15&window=14&threshold=2.5
Content-Type: multipart/form-data

Parameters:
  file: CSV with KPI data
  date: Anomaly date to analyze (YYYY-MM-DD)
  window: Rolling window for analysis (default: 14)
  threshold: Z-score threshold (default: 2.5)

Response:
{
  "date": "2024-01-15",
  "root_cause": {
    "drivers": {
      "primary_driver": "ad_spend",
      "primary_driver_pct_change": 25.5,
      "target_kpi": "revenue",
      "drivers_ranked": [
        {
          "factor": "ad_spend",
          "current_value": 5000,
          "baseline_value": 4000,
          "pct_change": 25.0,
          "correlation": 0.85
        },
        ...
      ]
    },
    "confidence": {
      "confidence": "High",
      "score": 0.85,
      "should_abstain": false,
      "reason": "Sufficient data points and strong correlation"
    },
    "multi_kpi_overlap": [...]
  },
  "decision_engine": {
```

```
    "primary_driver": "ad_spend",
    "recommendations": [
      {
        "action": "Review ad spend efficiency",
        "impact_score": 0.92,
        "reasoning": "Ad spend increased 25% but ROI decreased"
      },
      ...
    ]
  }
}
```

---

6. AI Narrative & Insights Endpoints

Generate Business Narratives

```
POST /narrative?kpi=revenue&date=2024-01-15&window=14&threshold=2.5
Content-Type: multipart/form-data

Parameters:
  file: KPI CSV
  kpi: KPI to analyze
  date: anomaly date (YYYY-MM-DD)
  window: rolling analysis window
  threshold: statistical threshold
  persona: optional persona identifier
```

The narrative endpoint runs the deterministic root-cause pipeline first and then turns the computed analysis into human-readable business explanations. The quantitative result comes from Python/database calculations; the LLM is used for explanation.

---

7. PDF Report Generation

Generate Complete PDF Report

The PDF report is generated from the **already-computed Root Cause analysis result** returned to the frontend.

The frontend sends the analysis result as `analysis_json` to `/report`, so downloading a report does not independently regenerate a second narrative.

```
POST /report?kpi=revenue&date=2024-01-15&window=14&threshold=2.5
Content-Type: multipart/form-data

Form fields:
  file: KPI CSV
  analysis_json: JSON returned by the Root Cause/Narrative analysis
```

PDF Report Includes:

1. Executive summary
2. KPI snapshot
3. Root cause / driver analysis
4. Product-level revenue drivers when available
5. Net-profit/loss snapshot when available
6. Confidence metrics
7. Affected KPIs
8. Anomaly visualization
9. Existing AI narratives supplied by the analysis
10. Supporting evidence
11. Ranked recommendations

The report renderer only formats the supplied analysis; it is not a second source of business calculations.

---

8. Business Actions & Recommendations

Get Candidate Actions for a KPI

```
GET /actions?kpi=revenue

Response:
{
  "kpi": "revenue",
  "actions": [
    {
      "id": "action_1",
      "title": "Optimize ad spend allocation",
      "description": "Reallocate budget to highest-ROI channels",
      "impact": "High",
      "effort": "Medium"
    },
    ...
  ]
}
```

Get Historical Action Scores (v5 Model Only - Requires 3000+ Feedbacks)

```
GET /actions/scores?kpi=revenue

Response:
{
  "action_1": {
    "title": "Optimize ad spend allocation",
    "historical_score": 0.87,
    "times_taken": 5,
    "average_outcome": "positive"
  },
  ...
}
```

⚠️ Only available after training v5 model with 3000+ analyst feedbacks.

---

9. Feedback & Learning Loop

Submit Analyst Feedback

```
POST /feedback
Content-Type: application/json

Body:
{
  "kpi": "revenue",
  "anomaly_date": "2024-01-15",
  "root_cause": "ad_spend",
  "recommended_action": "Optimize ad spend allocation",
  "analyst_rating": 4,
  "action_taken": true,
  "outcome": "positive",
  "outcome_value": 5000.00,

  "primary_driver_pct_change": 25.5,
  "confidence_score": 0.85,

  "visitors_change": -5.2,
  "orders_change": 12.3,
  "revenue_change": 8.7,
  "aov_change": -3.5,
  "cac_change": 2.1,
  "ad_spend_change": 25.0
}

Response:
{
  "status": "success",
  "message": "Feedback stored successfully",
  "decision_id": 42
}
```

Retrieve Stored Feedback

```
GET /feedback?kpi=revenue&limit=50

Response:
{
  "count": 42,
  "feedback": [
    {
      "id": 42,
      "kpi": "revenue",
      "anomaly_date": "2024-01-15",
      "recommended_action": "Optimize ad spend allocation",
      "analyst_rating": 4,
      "action_taken": true,
      "outcome": "positive",
      "outcome_value": 5000.00,
      "confidence_score": 0.85,
      "created_at": "2024-01-16T10:30:00"
    },
    ...
  ]
}
```

---

10. Chat Interface

Chat with BID

```
POST /chat
Content-Type: multipart/form-data
```

The conversational assistant can use deterministic tools for:

- anomaly/root-cause analysis
- ranked recommendations
- supporting customer evidence
- historical feedback

For business-data questions, Python and PostgreSQL remain the quantitative source of truth.

PDF Q&A

The React chat drawer includes a **+** attachment button. A user can attach a PDF and ask questions about that document.

The backend uploads the PDF to Google Gemini and uses the configured Gemini model for document understanding.

Required environment variables:

```
GEMINI_API_KEY=your_gemini_api_key
GEMINI_MODEL=gemini-3.6-flash
```

The Gemini API key stays on the backend and is never placed in the frontend.

Dynamic KPI Support

The frontend and analysis windows are designed to work from the KPI columns actually present in the uploaded CSV. A business does not need to provide every KPI used by the demo dataset.

For example, a dataset may contain:

```
date,revenue,orders
```

without `cac`, `aov`, or `conversion_rate`. The UI should expose only the available analyzable KPIs.

📖 Usage Guide

Workflow for Basic Analysis (No ML Training Required)

Step 1: Load Data

```
# Upload raw order data
curl -X POST "http://localhost:8000/upload-orders" \
  -F "file=@orders.csv"

# Upload marketing metrics
curl -X POST "http://localhost:8000/upload-marketing" \
  -F "file=@marketing.csv"

# Upload customer reviews (for evidence)
curl -X POST "http://localhost:8000/upload-reviews" \
  -F "file=@reviews.csv"
```

Step 2: Calculate KPIs

```
curl -X POST "http://localhost:8000/calculate-kpis"
```

Step 3: Detect Anomalies

```
# Strong detection (Z-Score + Prophet)
curl -X POST "http://localhost:8000/detect-strong?kpi=revenue" \
  -F "file=@kpi_data.csv"
```

Step 4: Analyze Root Causes

```
curl -X POST "http://localhost:8000/root-cause?date=2024-01-15" \
  -F "file=@kpi_data.csv"
```

Step 5: Generate Report with Narratives

```
curl -X POST "http://localhost:8000/narrative?date=2024-01-
15&persona=marketing_manager" \
  -F "file=@kpi_data.csv"
```

Step 6: Generate PDF Report

The web application sends the completed Root Cause/Narrative response as `analysis_json` so the PDF uses the same analysis already shown to the user.

```
Root Cause/Narrative result
        ↓
frontend Download PDF
        ↓
POST /report with analysis_json
        ↓
PDF renderer
```

Step 7: Submit Feedback (for Learning)

```
curl -X POST "http://localhost:8000/feedback" \
  -H "Content-Type: application/json" \
  -d '{
    "kpi": "revenue",
    "anomaly_date": "2024-01-15",
    "recommended_action": "Optimize ad spend allocation",
    "analyst_rating": 4,
    "action_taken": true,
    "outcome": "positive"
  }'
```

🤖 Advanced: Model Training with recommendation_engine_v5.py

⚠️ IMPORTANT: Feedback Requirements

The v5 recommendation model benefits from a sufficiently large and representative feedback history. A small or new installation should not fail just because historical feedback is missing; the system should use its fallback/default recommendation scoring until enough real feedback has been collected.

Step 1: Seed Sample Feedback Data (For Testing Only)

If you want to test the v5 model with sample data before collecting real feedbacks:

```
python seedfeedback.py
```

This script will:

- Create 3000+ sample feedback records in `business_decisions` table
- Cover various KPI anomalies, root causes, and outcomes
- Generate realistic action effectiveness patterns
- Provide training data for the ML model to learn from

Note: In production, feedback comes from the `/feedback` API endpoint as users analyze anomalies.

Step 2: Ensure Feedback Table is Initialized

```
python database.py --init
```

This creates the `business_decisions` table needed for ML training.

Step 3: Accumulate Feedbacks

After using the platform for analysis, users submit feedback via the `/feedback` endpoint. The system learns patterns from these feedbacks.

Step 4: Train the v5 Model

Once you have 3000+ feedbacks:

```
# The v5 model automatically trains when called with sufficient data
GET /actions/scores?kpi=revenue
```

The model will:

- Analyze patterns from 3000+ historical feedbacks
- Learn which actions have been most effective
- Identify success patterns by KPI, root cause, and context
- Rank recommendations based on learned patterns

Step 5: Use Trained Recommendations

After training, recommendations are ranked by effectiveness:

```
GET /actions/scores?kpi=revenue

Response:
{
  "action_1": {
    "title": "Optimize ad spend allocation",
    "historical_score": 0.87,    # Effectiveness score
    "times_taken": 25,           # Number of times used
    "average_outcome": "positive" # Average outcome
  },
  "action_2": {
    "title": "Improve website performance",
    "historical_score": 0.92,
    "times_taken": 18,
    "average_outcome": "positive"
  }
}
```

⚙ Configuration

Environment Variables

Create a `.env` file in the root directory with:

```
# ============ DATABASE CONFIGURATION ============
DB_HOST=localhost
DB_NAME=business_Ai
DB_USER=postgres
DB_PASSWORD=your_secure_password
DB_PORT=5432

# ============ LLM CONFIGURATION ============
# Google Gemini API for AI narratives and insights
GEMINI_API_KEY=your_gemini_api_key_here

# ============ API CONFIGURATION ============
# IMPORTANT: Use GROQ_API_KEY (this is the primary API key for external callers)
GROQ_API_KEY=your_groq_api_key_for_external_callers

# ============ LOGGING & DEBUG ============
DEBUG=False
LOG_LEVEL=INFO
```

Report Customization

Modify styling in `report_pdf.py`:

```
# Title style
TITLE_FONT_SIZE = 24
TITLE_COLOR = (0, 0, 0)  # RGB

# Table styling
TABLE_HEADER_COLOR = (0.2, 0.4, 0.8)  # Dark blue
TABLE_ROW_COLOR = (0.95, 0.95, 0.95)  # Light gray

# Body text
BODY_FONT_SIZE = 11
LINE_SPACING = 1.5
```

📦 Modules

`main.py` — FastAPI Application (Entry Point)

Purpose: Core API server exposing all endpoints

What it does:

- Starts Uvicorn ASGI server on port 8000
- Auto-generates Swagger UI at `/docs`
- Exposes 30+ endpoints for data upload, anomaly detection, analysis, and reporting
- Manages CORS for frontend communication

How to Run:

```
uvicorn main:app --reload --port 8000
```

`Create_inputDataBase.py` — Database Management (Core Tables)

Purpose: Database initialization and connection management for core KPI tables

Key Functions:

- `get_connection()`: Establishes PostgreSQL connection
- `create_tables()`: Initializes schema with RawData, MarketingData, Kpis tables
- `main()`: CLI interface for database operations

Usage:

```
# Test connection
python Create_inputDataBase.py

# Initialize database
python Create_inputDataBase.py --init
```

`database.py` — Database Utilities & Feedback Management

Purpose: Database utilities and feedback table management for v5 ML model training

Features:

- Connection pooling
- Error logging
- Transaction management
- Feedback table initialization

Usage:

```
# Initialize feedback table for v5 model training
python database.py --init
```

`seedfeedback.py` — Feedback Data Seeding

Purpose: Populate sample feedback data for testing the v5 model

What it does:

- Creates 3000+ sample feedback records in `business_decisions` table
- Covers various KPI anomalies and root causes
- Generates realistic action outcomes and effectiveness scores
- Provides training data for the ML recommendation engine

Usage:

```
python seedfeedback.py
```

When to Use:

- For testing v5 model in development
- To understand feedback data structure
- NOT needed for production (feedback comes via `/feedback` API)

`anomaly_detector.py` — Statistical Anomaly Detection

Purpose: Detect anomalies using Z-score method

Algorithm:

1. Calculates 14-day rolling mean and standard deviation
2. Computes Z-score for each day: `(value - mean) / std`
3. Flags days with |Z-score| > threshold (default: 2.5) as anomalies

Key Functions:

- `detect_anomalies_zscore()`: Single KPI anomaly detection
- `detect_anomalies_multi()`: Multiple KPI anomaly detection

`prophet_detector.py` — Time-Series Forecasting

Purpose: Detect anomalies using Facebook's Prophet

Algorithm:

1. Trains Prophet model on historical data
2. Generates forecasts with confidence intervals
3. Flags days outside confidence interval as anomalies
4. Combines results with Z-score detection (OR logic)

Key Functions:

- `detect_anomalies_ensemble()`: Combines Z-score + Prophet for stronger detection

`root_cause.py` — Root Cause Analysis

Purpose: Identify which KPI or factor caused the anomaly

Analysis Steps:

1. Identify Primary Driver: Which KPI changed most on anomaly date
2. Factor Correlation: Analyze correlation between drivers
3. Baseline Comparison: Compare to 14-day baseline
4. Calculate % Changes: Quantify impact
5. Confidence Scoring: Assess reliability

`product_drivers.py` — Product-Level Analysis

Purpose: Explain revenue movement using product-level contribution analysis.

Features:

- Compares each product with its trailing baseline
- Calculates revenue change and contribution percentage
- Provides volume and price effects
- Flags sparse product history
- Computes an on-demand net-profit/loss snapshot from order and production-cost data

---

`action_engine.py` — Business Action Generation

Purpose: Generate candidate actions based on root cause

Action Categories:

- Pricing Actions: Adjust pricing, run promotions
- Marketing Actions: Optimize ad spend, change targeting
- Operational Actions: Scale resources, process improvements
- Product Actions: Feature updates, quality improvements

`recommendation_engine_v5.py` — ML-Based Action Ranking

⚠️ **REQUIRES: 3000+ analyst feedbacks for proper training**

Purpose: Rank business actions using ML model trained on historical feedback

What it does:

1. Loads feedback from `business_decisions` table
2. Extracts features from anomalies, root causes, and outcomes
3. Trains scikit-learn model on action effectiveness
4. Ranks recommendations based on learned patterns

Usage:

```
# The model trains automatically when called with sufficient data
(3000+)
GET /actions/scores?kpi=revenue
```

`llm_narrative.py` — AI Narrative Generation

Purpose: Generate human-readable business insights using LLM

Personas:

1. **Marketing Manager**: Focus on marketing metrics, campaign effectiveness, ROI
2. **Sales Ops Manager**: Focus on operational efficiency, pipeline, conversion

Features:

- Uses Google Gemini API for generation
- Incorporates customer evidence and context
- Structured output with business factors

`text_retrieval.py` — RAG / Evidence Retrieval

Purpose: Find supporting evidence from customer reviews/feedback

Algorithm:

1. Parses root cause analysis
2. Searches for related keywords in reviews
3. Ranks by relevance to anomaly date and factors
4. Returns top K matching reviews

`report_pdf.py` — PDF Report Generation

Purpose: Create professional PDF reports with all analysis results

Report Sections:

1. Title Page
2. Executive Summary

3. Root Cause Analysis Table
4. Confidence Metrics
5. Affected KPIs
6. KPI Trend Graph
7. AI Narratives (Multiple Personas)
8. Supporting Evidence
9. Recommended Actions

Libraries Used:

- ReportLab: PDF generation
- Matplotlib: Visualization
- Pandas: Data manipulation
- Pillow: Image processing

`chatbot.py` — Conversational AI

Purpose: Enable Q&A about anomalies via chat interface

Features:

- Conversation history management
- Context from KPI data and reports
- PDF document understanding
- Multi-turn conversations

`business_config.py` — Configuration Management

Purpose: Centralized business configuration

Contents:

- KPI definitions
- Default analysis parameters
- Persona configurations

📄 Report Generation

Report Sections In Detail

1. Executive Summary

- What anomaly was detected
- Primary driver of the anomaly
- Confidence in the analysis
- Top recommendation

2. Root Cause Analysis Table

Driver	Current	Baseline	% Change	Correlation
ad_spend	$5,000	$4,000	+25%	0.85
website_visits	45,000	50,000	-10%	0.72

3. Confidence Metrics

- Confidence Level: High / Medium / Low
- Confidence Score: 0-1 numerical score
- Data Quality assessment

4. KPI Trend Graph

- KPI line chart over time
- Baseline/trend line
- Anomalies marked with X

5. AI Narratives (per Persona)

Human-readable explanations from:

- Marketing Manager: Focus on marketing metrics and ROI
- Sales Ops Manager: Focus on operational efficiency

6. Supporting Evidence

Customer review snippets:

- Date, verbatim feedback, relevance score, sentiment

7. Recommendations

Ranked action list with:

- Action description
- Rationale
- Impact Score (0-1)
- Effort (Low/Medium/High)
- Timeline

📝 Contributing

1. **Fork the repository**

```
git clone https://github.com/YOUR_USERNAME/BusinessIntelligence.AI---BID.git
```

2. **Create a feature branch**

```
git checkout -b feature/amazing-feature
```

3. **Make your changes**
   - Follow PEP 8 style guidelines for Python
   - Add docstrings to new functions
   - Test thoroughly
4. **Commit your changes**

```
git commit -m 'Add amazing feature: description'
```

5. **Push to your branch**

```
git push origin feature/amazing-feature
```

6. **Open a Pull Request**

- Provide clear description of changes
- Reference any related issues
- Include examples if applicable

📄 License

This project is open source. See LICENSE file for details.

🤝 Support

For issues, questions, or suggestions:

1. **Check existing GitHub issues** for solutions
2. **Open a new GitHub issue** with:
   - Clear title describing the problem
   - Steps to reproduce
   - Expected vs. actual behavior
   - Error messages/logs
3. **Contact the maintainers** via email

🎯 Roadmap

The architecture is designed for progressive expansion across enterprise integration, governance, scale and advanced intelligence.

Future Enhancements:

- [ ] Multi-language support for narratives
- [ ] Custom anomaly detection algorithms
- [ ] Real-time dashboard with WebSocket updates
- [ ] Integration with Slack/Teams for alerts
- [ ] Advanced time-series decomposition
- [ ] Automated report scheduling
- [ ] User role management and permissions

- [ ] Enhanced ML model with deep learning

Last Updated: August 2026
Author: Giridhar692005
Repository: GitHub